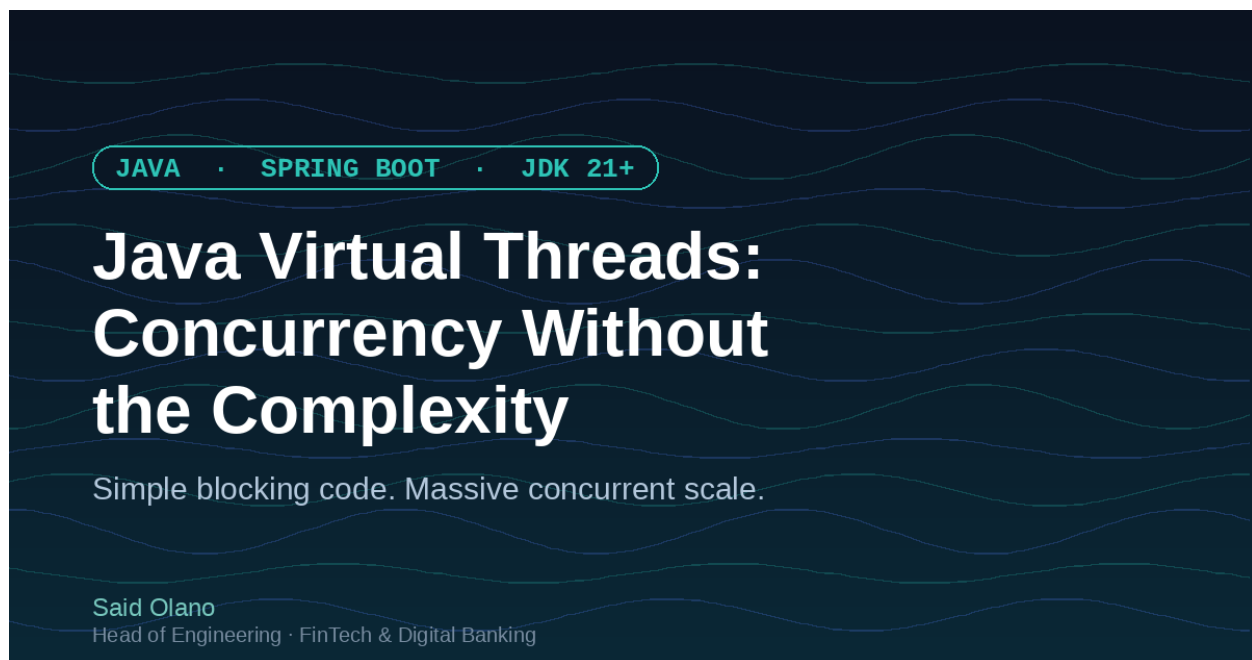




Java Virtual Threads: Concurrency sin la Complejidad

Por Said Olano - Head of Engineering, FinTech & Digital Banking

Si has construido servicios en Java durante un tiempo, conoces bien esta disyuntiva: un hilo por petición (thread-per-request) es simple, pero no escala más allá de unos pocos miles de conexiones concurrentes; y la programación reactiva (Project Reactor, Mono/Flux, cadenas de callbacks) escala de maravilla, pero convierte tu código en algo que solo la mitad del equipo puede depurar.

JDK 21 nos da una tercera opción: los Virtual Threads (hilos virtuales), entregados a través de Project Loom. Te permiten seguir escribiendo Java simple y bloqueante, mientras la JVM se encarga de la gimnasia de planificación que antes requería un framework reactivo.

El problema de la concurrencia tradicional

Un hilo de plataforma (del sistema operativo) es costoso. Cada uno reserva alrededor de 1MB de espacio de pila e involucra al sistema operativo en cada cambio de contexto. Si levantas un hilo por cada petición entrante, el servidor colapsa en algún punto entre unos pocos miles y diez mil conexiones concurrentes — mucho antes de que la CPU misma sea el cuello de botella.

La solución tradicional ha sido el código asíncrono y no bloqueante: streams reactivos, callbacks, cadenas de `CompletableFuture`. Funcionan, pero cambian la forma en que escribes y lees código, cómo depuras stack traces y cómo razonas sobre los fallos.

Entran los hilos virtuales

Un hilo virtual es un hilo ligero gestionado por la JVM en lugar del sistema operativo. Puedes crear millones de ellos, porque no se mapean 1:1 a hilos del sistema operativo — la JVM multiplexa muchos hilos virtuales sobre un pequeño grupo de hilos "portadores" (carrier threads), estacionando un hilo virtual cada vez que se bloquea en I/O y liberando el hilo portador para que atienda a otro.

El resultado: escribes código bloqueante ordinario —

```
public String fetchUser(String id) {  
    var response = httpClient.send(request, BodyHandlers.ofString());  
    return response.body();  
}
```

— y el runtime te da las características de rendimiento del código asíncrono, sin la sobrecarga mental.

Activándolo en Spring Boot

Desde Spring Boot 3.2, activar los hilos virtuales para tu servidor web y tus ejecutores de tareas es un cambio de una sola línea:

```
spring.threads.virtual.enabled=true
```

Y ya está. Tus endpoints de `@RestController`, las llamadas JDBC (con un driver JDBC 4.3+) y las llamadas de `RestClient`/`RestTemplate` ahora corren sobre hilos virtuales por defecto. Sin reescribir código, sin tipos reactivos filtrándose en tu capa de servicio.

Dónde brillan

Los hilos virtuales están hechos para cargas de trabajo dominadas por I/O: APIs REST que llaman a servicios río abajo, consultas a bases de datos, acceso a archivos, cualquier cosa que pase la mayor parte del tiempo esperando en lugar de calculando. Este es exactamente el perfil de la mayoría de los microservicios de Spring Boot.

Dónde no ayudan

No son una bala de plata para trabajo intensivo en CPU. Ejecutar un hilo virtual no hace que una multiplicación de matrices o una serialización JSON sea más rápida — sigues teniendo el mismo número de núcleos de CPU. Si tu servicio es intensivo en cómputo, los hilos virtuales no moverán mucho la aguja.

La trampa: el "pinning" de hilos

Los hilos virtuales pueden quedar "clavados" (pinned) a su hilo portador en un par de situaciones — sobre todo dentro de un bloque o método `synchronized` que bloquea. Mientras está clavado, el hilo portador no puede reutilizarse, lo cual anula el propósito. La solución suele ser sencilla: reemplazar `synchronized` por `java.util.concurrent.locks.ReentrantLock` en las rutas calientes que corren sobre hilos virtuales. (JDK 24 elimina la mayor parte de esta limitación, pero si estás en la LTS 21, tenlo presente.)

Para cerrar

Los hilos virtuales son una de las pocas funcionalidades de la JVM que te dan una ganancia real de escalabilidad por el costo de una bandera de configuración y un poco de atención a los casos límite. Si corres Spring Boot sobre JDK 21+, vale la pena probarlo en un entorno de staging antes de tu próxima prueba de carga.

¿Ya adoptaste los hilos virtuales en un servicio de Spring Boot en producción? Me encantaría saber qué te encontraste — déjalo en los comentarios.